

Coleções em Python Sets e dicionários



Coleções em Python

Existem quatro tipos de dados de coleção em Python:

Listas

☒	<u>Ordenada</u>
☐	<u>Mutável</u>
☐	<u>Aceita duplicados</u>

Tuplas

☒	<u>Ordenada</u>
☐	<u>Imutável</u>
☐	<u>Aceita duplicados</u>

Sets

☒	<u>Desordenada</u>
☐	<u>'Imutável'*</u>
☐	<u>Nenhum duplicado</u>

Dicionários

☒	<u>Ordenada **</u>
☐	<u>Mutável</u>
☐	<u>Nenhum duplicado</u>

***Set imutável:** Não pode ser alterado mas, aceita remoção e adição de itens.

****Dicionário Ordenado:** A partir da versão 3.7 do Python, os dicionários são *ordenados* .
No Python 3.6 e anteriores, os dicionários não são *ordenados* .

Sets



Tipos Sequenciais – Sets

Vamos conhecer um pouco sobre a coleção **Sets** 😊

- Uma SET tem característica de desordenação, ou seja, não tem índice específico para os dados.
- Um set é criado utilizando chaves `{ }` ou a função `set()`.

Declarando uma set

```
listaSet = { 4, 'texto', True, 2.5 }  
  
print(listaSet)  
print(type(listaSet))
```

Console

```
{True, 4, 2.5, 'texto'}  
  
<class 'Set'>
```

Como declarar um set?

Declarando uma set

```
listaSet = { }  
  
print(listaSet)  
print(type(listaSet))
```

Console

```
{ }  
  
<class 'dict'>
```

O Python identifica que é um dicionário e não uma SET

O que fazer para deixar uma set não inicializada?

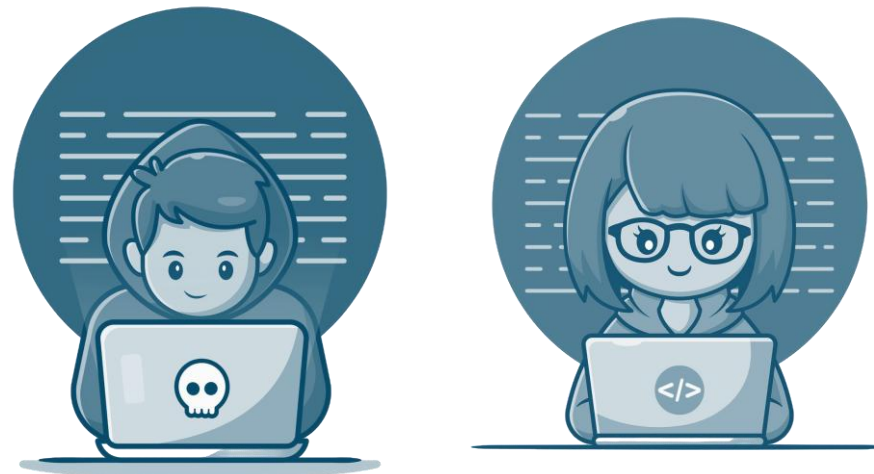
Usando a função
`set()`

Console

```
listaSet = set()  
  
print(type(listaSet))
```

```
<class 'set'>
```

- Cria uma lista set sem inicialização, comando: `set()`
- Adiciona um elemento na coleção, comando: `add()`
- Remove um elemento da coleção, comando: `remove()`
- Limpa todo o conjunto (o esvazia), comando: `clear()`
 - O comando `clear()` também pode ser aplicado em lista.



Exemplos

```
# Criando um conjunto (set) de frutas
frutas = {'maçã', 'banana', 'laranja', 'maçã'}

print(frutas)
# Saída: {'banana', 'laranja', 'maçã'}
# 'maçã' aparece apenas uma vez, pois sets não permitem valores duplicados.
```

add()

```
frutas.add('morango')
print(frutas)

# Saída: {'banana', 'laranja', 'maçã', 'morango'}
```

remov()

```
frutas.remove('banana')
print(frutas)

# Saída: {'maçã', 'laranja', 'morango'}
```

clear()

```
frutas.clear() # Limpa todos os elementos do set
print(frutas)

# Saída: set()
```

Método add()

```
listaSet = set()

for recebe in range(3):
    nome = input('Nome: ')
    numero = int(input('Número: '))

    listaSet.add(nome)
    listaSet.add(numero)

print(listaSet)
```

```
print(listaSet)
```

```
print(listaSet)
```

Console

```
Nome: Renata
Número: 6
Nome: Maria
Número: 9
Nome: Renata
Número: 12
{'Maria', 6, 9, 12, 'Renata'}
```

Características do SET

Declarando uma set com valores repetidos

```
listaSet = { 4, 'texto', 4, 4}
```

```
print(listaSet)
```

```
print(type(listaSet))
```

Console

```
{ 'texto', 4 }
```

```
<class 'Set'>
```

Características do SET

Os conjuntos não são ordenados, portanto, você não pode ter certeza em qual ordem os itens aparecerão.

Declarando uma set

```
listaSet = { 1,2,3,4,5,6}  
print(listaSet)
```

Descobrendo seu tamanho

```
listaSet = len(listaSet)  
print(listaSet)
```

Console

```
{ 3,5,6,1,4,2}  
  
6
```

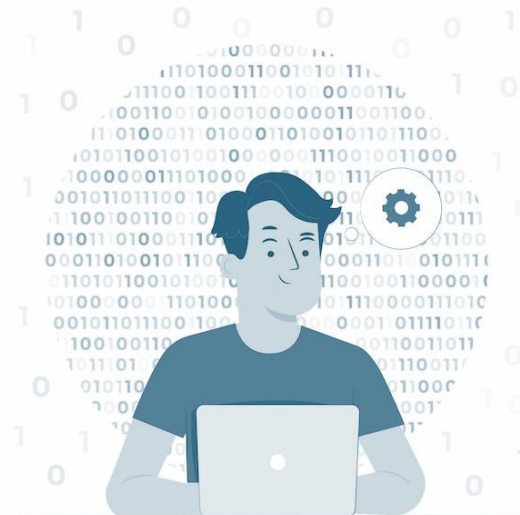
O que acontece se eu adicionar um elemento que já existe no SET?

Será desconsiderado!

O que acontece se eu remover um elemento que não existe no SET?

Dará um erro de chave!

KeyError



```
listaSet = {"maça", "banana", "uva"}  
print("banana" in listaSet)
```



O retorno será um booleano : **True**

Dicionários



Porque usar Dicionário?

VAMOS LÁ!

- Quero fazer uma lista de contatos telefônicos em Python. Para isso, preciso armazenar os números dos contatos. A princípio, colocaremos em uma lista.

```
telefones = ['1234-5678', '9999-9999', '8765-4321', '8877-7788']
```

Lista de contato

- Temos os números de telefone armazenados. Mas... qual o sentido de termos uma lista de números sem saber de quem é?

Daí que surge o dicionário!

- Com o dicionário irá ser possível que seus contatos tenham o nome e o telefone conectados, assim, ficará mais fácil você lembrar qual telefone é referente aos contatos.



O que é um Dicionário?

- Um **dicionário** em Python é uma estrutura de dados que armazena pares de chave-valor. Ele é amplamente utilizado para organizar dados de forma que você possa acessar facilmente um valor associado a uma chave específica.
- Sua sintaxe básica é: `{'chave' : 'valor'}`. Utiliza-se `{ }` para delimitar o dicionário e a chave é separada do valor por dois pontos `:`.

```

pessoa = {
    "nome": "Alice",
    "idade": 30,
    "cidade": "São Paulo"
}
```


'chave' 'valor'

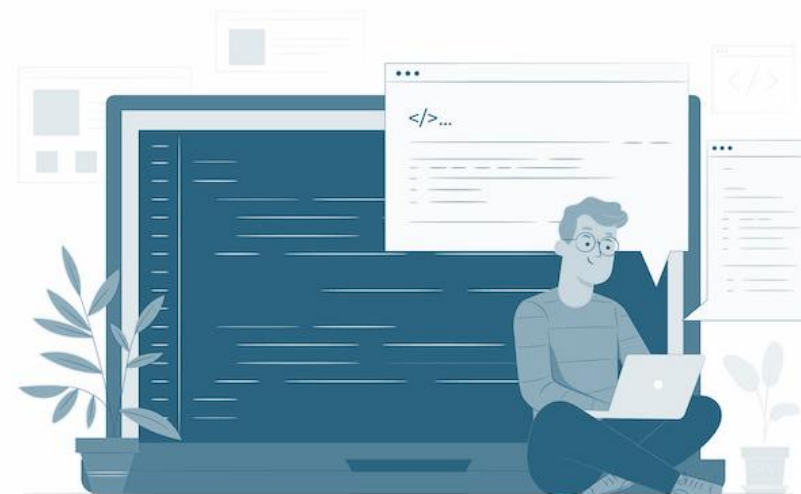
Características de um Dicionário:

Não Ordenado: Os dicionários não mantêm a ordem dos elementos. Desde o Python 3.7, a ordem de inserção é preservada, mas isso não é uma característica principal da estrutura.

Mutável: Você pode modificar, adicionar ou remover pares chave-valor após a criação do dicionário.

Acesso Rápido: Os valores podem ser acessados rapidamente utilizando suas chaves, o que torna os dicionários muito eficientes para busca de dados.

Chaves Únicas: Cada chave em um dicionário deve ser única. Se uma chave já existir, seu valor pode ser atualizado.



Criando um dicionário sem inicialização

```
dicionario = dict()
```

ou

```
dicionario = { }
```

```
print(type(dicionario))
```

Console

```
<class 'dict'>
```

Inicializando um dicionário

```
dicionario = {  
    'Yan': 1234-5678,  
    'Carla': 23333  
}  
print(dicionario)
```

Criando um dicionário sem inicialização

Exemplo de dicionário retornando apenas o valor

```

pessoa = {
    "nome": "Alice",
    "idade": 30,
    "cidade": "São Paulo"
}

print(pessoa["nome"]) # Imprime: Alice
print(pessoa["idade"]) # Imprime: 30
print(pessoa["cidade"]) # Imprime: São Paulo

print(f'O {pessoa["nome"]} tem {pessoa["idade"]} anos.')
# Imprime: O Alice tem 30 anos.
```

Chama os valores
por **colchetes**

As chaves de um dicionário são textos.
Por isso, ao acessá-las pelos colchetes,
devemos colocá-las entre **aspas**.

Como criar um dicionário Python?

Criar um dicionário em Python é simples e envolve o uso de chaves `{ }` para encapsular os pares de chave-valor.

Você cria a variável (nome do dicionário) e define entre chaves os pares de chave-valor, com a sintaxe **“chave”: valor**, separando cada chave-valor por vírgula. As chaves são definidas entre aspas e são separadas do valor por dois pontos.

Exemplo de
Criação

```
dicionario = {"chave": 'valor', "chave2": 'valor2', "chave3": 'valor3'}
```

```
emails_shopping = {  
    "recife": "shoppingrecife@gmail.com",  
    "tacaruna": "tacaruna@gmail.com",  
    "guararapes": "guararapes@gmail.com"  
}
```

Exemplo de Dicionário:

```
# Criando um dicionário
pessoa = {
    "nome": "Alice",
    "idade": 30,
    "cidade": "São Paulo"
}

# Acessando valores
print(pessoa["nome"]) # Output: Alice
print(pessoa["idade"]) # Output: 30

# Adicionando um novo par chave-valor
pessoa["profissão"] = "Engenheira"

# Atualizando um valor existente
pessoa["idade"] = 31

# Removendo um par
del pessoa["cidade"]

print(pessoa)
```

```
print(pessoa)
```

E podemos ir além:

```
# Criando um dicionário aninhado
empresa = {
    "nome": "Tech Solutions",
    "funcionários": [
        {"nome": "Alice", "cargo": "Engenheira"},
        {"nome": "Bob", "cargo": "Designer"}
    ]
}

# Acessando dados do dicionário aninhado
print(empresa["funcionários"][0]["nome"]) # Saída: Alice
print(empresa["funcionários"][1]["cargo"]) # Saída: Designer
```

```
print(empresa["funcionários"][1]["cargo"]) # Saída: Designer
print(empresa["funcionários"][0]["nome"]) # Saída: Alice
```

Percorrendo um Dicionário

Podemos usar o **for** para percorrer as **chaves** ou os **valores** ou **ambos** em um dicionário.

```
# Criando um dicionário de frutas e suas cores
frutas_cores = {
    'maçã': 'vermelha',
    'banana': 'amarela',
    'laranja': 'laranja'
}

# Percorrendo chaves e valores
for fruta, cor in frutas_cores.items():
    print(f'A {fruta} é {cor}.')
```

```
print(f'A {fruta} é {cor}.')
for fruta, cor in frutas_cores.items():
```

Console

A maçã é vermelha.

A banana é amarela.

A laranja é laranja.

Como verificar todos os valores – values()

Método values()

```
peessoas = {  
    'nome': 'Ana',  
    'sexo': 'F',  
    'idade': 22}  
  
print(peessoas)  
print(peessoas.values())
```

Console

```
{'nome': 'Ana', 'sexo': 'F',  
 'idade': 22}  
  
dict_values(['Ana', 'F', 22])
```

Como verificar todos as chaves – keys()

Método `keys()`

```
peessoas = {  
    'nome': 'Ana',  
    'sexo': 'F',  
    'idade': 22}  
  
print(peessoas.keys())
```

Console

```
dict_keys(['nome', 'sexo',  
           'idade'])
```

Como verificar as chaves e valores juntas – items()

Método `items()`



```
peessoas = {  
    'nome': 'Ana',  
    'sexo': 'F',  
    'idade': 22  
}  
print(peessoas.items())
```

Console

```
dict_items([('nome', 'Ana'), ('sexo', 'F'), ('idade', 22)])
```

Coleções em Python - Dicionários

Retornando o
dicionário por um
laço de repetição

for

values() - valores

keys() - chaves

items() - ambos

```
peessoas = {  
    'nome': 'Ana',  
    'sexo': 'F',  
    'idade': 22  
}  
  
# Recebendo apenas os valores  
for valor in pessoas.values():  
    print(valor) # Saída: Ana / F / 22  
  
# Recebendo apenas as chaves  
for chave in pessoas.keys():  
    print(chave) # Saída: nome / sexo / idade  
  
# Recebendo chaves e valores  
for chave, valor in pessoas.items():  
    print(f"{chave}: {valor}")  
  
# Saída: nome: Ana  
#         sexo: F  
#         idade: 22
```

```
#         idade: 22
```

Método – get()

O método **get()** é utilizado para buscar um valor em um dicionário através da sua chave.

Como a chave "**abacaxi**" não existe no dicionário, o método retorna **None**.

Como a chave "**uva**" não existe no dicionário, o método retorna o valor padrão informado: "**Desconhecida**".

```
# Criando um dicionário de frutas e suas cores
frutas_cores = {
    'maçã': 'vermelha',
    'banana': 'amarela',
    'laranja': 'laranja'
}           chave           valor

# Acessando uma chave que existe
cor_maca = frutas_cores.get('maçã')

print(f'Cor da maçã: {cor_maca}')
# Saída: vermelha

# Acessando uma chave que não existe, sem valor padrão
cor_abacaxi = frutas_cores.get('abacaxi')

print(f'Cor do abacaxi: {cor_abacaxi}')
# Saída: None

# Acessando uma chave que não existe, com valor padrão
cor_uva = frutas_cores.get('uva', 'Desconhecida')

print(f'Cor da uva: {cor_uva}')
# Saída: Desconhecida
```

Modificando, Adicionando e removendo

Modificando:

Como a chave "**banana**" já existe no dicionário, seu valor é atualizado para "**verde**".

Adicionando:

Como a chave "**morango**" não existe no dicionário, um novo par chave-valor é adicionado.

Removendo com `pop()`:

Como a chave "**laranja**" existe no dicionário, o método remove o item e retorna o valor associado: "**laranja**".

Removendo com `del`:

Como a chave "**maçã**" existe no dicionário, o comando remove completamente o par chave-valor.

```
# Dicionário inicial
frutas_cores = {
    'maçã': 'vermelha',
    'banana': 'amarela',
    'laranja': 'laranja'
}

# Modificando o valor associado à chave 'banana'
frutas_cores['banana'] = 'verde'

# Adicionando um novo par chave:valor
frutas_cores['morango'] = 'vermelho'

print(frutas_cores)
# Saída:
# {'maçã': 'vermelha',
#  'banana': 'verde',
#  'laranja': 'laranja',
#  'morango': 'vermelho'}

# Removendo um item com pop()
cor_laranja = frutas_cores.pop('laranja')

print(cor_laranja)
# Saída: laranja

# Usando del para remover um item
del frutas_cores['maçã']

print(frutas_cores)

# Saída:
# {'banana': 'verde', 'morango': 'vermelho'}
```

Verificando Existência de Chaves e Valores - in

Antes de acessar um valor em um dicionário, é importante verificar se a **chave** existe.

```
emails_shopping = {  
    "recife": "shoppingrecife@gmail.com",  
    "tacaruna": "tacaruna@gmail.com",  
    "guararapes": "guararapes@gmail.com"  
}  
  
if "tacaruna" in emails_shopping:  
    print("Tem sim")  
else:  
    print("Tem não")
```

Tem sim

Também é possível verificar se um valor específico existe entre os **valores** do dicionário.

```
if "tacaruna@gmail.com" in emails_shopping.values():  
    print("O e-mail está presente no dicionário.")
```

Exercícios



Exercício 1

Objetivo: Praticar a criação e consulta de dados em um dicionário.

Descrição:

Crie um programa que:

- Armazene o preço de 3 produtos em um dicionário.
- Depois, solicite ao usuário o nome de um produto e exiba seu preço.
- Caso o produto não exista no dicionário, exiba a mensagem de produto não cadastrado.

RESULTADO ESPERADO

```
Digite o nome do produto: arroz
Preço: R$ 8.50
```

```
Digite o nome do produto: Ovo
Produto não encontrado.
```

Exercício 1

```
produtos = {  
    'arroz': 8.50,  
    'feijao': 9.90,  
    'macarrao': 5.75  
}  
  
produto = input('Digite o nome do produto: ').lower()  
  
if produto in produtos:  
    print(f'Preço: R$ {produtos[produto]:.2f}')  
else:  
    print('Produto não encontrado.')
```

```
        print('Produto não encontrado.')
```

RESULTADO ESPERADO

```
Digite o nome do produto: arroz  
Preço: R$ 8.50
```

```
Digite o nome do produto: Ovo  
Produto não encontrado.
```

Exercício 2

Objetivo: Praticar alteração de valores em dicionários.

Descrição:

- Crie um dicionário contendo os seguintes clientes e saldos:
 - Maria = 5000
 - João = 3200
 - Ana = 7500
- Solicite o nome do cliente e o novo saldo.
- Atualize o saldo informado e exiba o cadastro atualizado.

RESULTADO ESPERADO

```
Cliente: Maria  
Novo saldo: 6000  
Saldo atualizado com sucesso!  
{'Maria': 6000.0, 'João': 3200, 'Ana': 7500}
```

```
Cliente: Renata  
Cliente não encontrado.
```

Exercício 2

```
clientes = {  
    'Maria': 5000,  
    'João': 3200,  
    'Ana': 7500  
}  
  
nome = input('Cliente: ').capitalize()  
  
if nome in clientes:  
    novo_saldo = float(input('Novo saldo: '))  
    clientes[nome] = novo_saldo  
    print('Saldo atualizado com sucesso!')  
    print(clientes)  
else:  
    print('Cliente não encontrado.')
```

RESULTADO ESPERADO

```
Cliente: Maria  
Novo saldo: 6000  
Saldo atualizado com sucesso!  
{'Maria': 6000.0, 'João': 3200, 'Ana': 7500}
```

```
Cliente: Renata  
Cliente não encontrado.
```

Exercício 3

Objetivo: Praticar alteração de valores em dicionários.

Descrição:

- Crie um dicionário contendo os seguintes clientes e saldos:
 - Maria = 5000
 - João = 3200
 - Ana = 7500
- Solicite o nome do cliente e o novo saldo.
- Atualize o saldo informado e exiba o cadastro atualizado.

RESULTADO ESPERADO

```
Cliente: Maria  
Novo saldo: 6000  
Saldo atualizado com sucesso!  
{'Maria': 6000.0, 'João': 3200, 'Ana': 7500}
```

```
Cliente: Renata  
Cliente não encontrado.
```

Exercício 3

```
clientes = {  
    'Maria': 5000,  
    'João': 3200,  
    'Ana': 7500  
}  
  
nome = input('Cliente: ').capitalize()  
  
if nome in clientes:  
    novo_saldo = float(input('Novo saldo: '))  
    clientes[nome] = novo_saldo  
    print('Saldo atualizado com sucesso!')  
    print(clientes)  
else:  
    print('Cliente não encontrado.')
```

RESULTADO ESPERADO

```
Cliente: Maria  
Novo saldo: 6000  
Saldo atualizado com sucesso!  
{'Maria': 6000.0, 'João': 3200, 'Ana': 7500}
```

```
Cliente: Renata  
Cliente não encontrado.
```

Exercício 4

Objetivo: Praticar inclusão de itens, percorrer dicionários e exibir dados.

Descrição:

- Crie um programa que cadastre 3 alunos.
- Para cada aluno, o usuário deverá informar:
 - Nome
 - Nota
- Armazene os dados em um dicionário.
- Ao final, exiba todos os alunos cadastrados e suas respectivas notas utilizando um laço for.

RESULTADO ESPERADO

```
Nome: Renata  
Nota: 10  
Nome: Eduarda  
Nota: 9.5  
Nome: Maria  
Nota: 8.75  
  
Alunos cadastrados:  
Renata - Nota: 10.0  
Eduarda - Nota: 9.5  
Maria - Nota: 8.75
```

```
Renata - Nota: 10.0  
Eduarda - Nota: 9.5  
Maria - Nota: 8.75
```

Exercício 4

```
alunos = {}

for i in range(3):
    nome = input('Nome: ')
    nota = float(input('Nota: '))

    alunos[nome] = nota

print('\nAlunos cadastrados:')

for nome, nota in alunos.items():
    print(f'{nome} - Nota: {nota}')
```

```
print(f'{nome} - Nota: {nota},')
for nome, nota in alunos.items():
```

RESULTADO ESPERADO

```
Nome: Renata
Nota: 10
Nome: Eduarda
Nota: 9.5
Nome: Maria
Nota: 8.75
```

```
Alunos cadastrados:
Renata - Nota: 10.0
Eduarda - Nota: 9.5
Maria - Nota: 8.75
```

```
Maria - Nota: 8.75
Eduarda - Nota: 9.5
Renata - Nota: 10.0
```